Activity No:	
Seatwrok 6.2 Files and Streams	
Course Code: CPE007	**Program:** Computer Engineering
Course Title: Programming Logic and Design	**Date Performed: October 22, 2026**
Section: CPE11S2	**Date Submitted: October 22, 2026**
Name(s): Zachary Truman A. Cortez	**Instructor: Sir Lusung**

Objectives

- **This activity aims to develop programming skills using the concepts of files and streams.**

Intended Learning Outcomes (ILOs):

1. Work with files
2. Read from a file
3. Write to a file
4. Process simple data
5. Understand the concept of the loop over file content

Instructions

Directions: Use Dev-C++ for the program and include comments for explanations.

1. Create a program that reads floating-point numbers line-by-line from a text file. If non-numeric characters are encountered on a line, log a warning, clear the error, and skip to the next line without crashing.

2. Write a program that opens a text file containing lines of mixed numbers and words. Read through the file line by line, print valid numbers, and handle/skip lines containing invalid text data without halting execution.

Submit a document with the format CPE007_Seatwork6_FullName.docx along with the program in two (2) .cpp file formats.

Output/Answer

1.
CODE:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
using namespace std;

int main()
{
   // Create an input file stream and try to open the file for reading
   ifstream inFile("numbers.txt");

   // If the file could not be opened (missing, wrong folder, etc.),
   // show an error message and stop the program with a non-zero code
   if (!inFile)
   {
      cout << "Error: could not open numbers.txt" << endl;
      return 1;
   }

   // Create an output file stream to record warnings in a log file
```

```
ofstream logFile("warnings.log");

string line;          // holds the current line read from the file
int lineNo = 0;       // tracks the current line number (for messages)
int validCount = 0;   // counts how many valid numbers were found
double sum = 0.0;     // running total of all valid numbers

// LOOP OVER FILE CONTENT:
// getline reads one whole line into 'line'. It returns false when
// the end of the file is reached, which ends the loop automatically.
while (getline(inFile, line))
{
   lineNo++;   // we are now on the next line of the file

   // If the line is empty, skip it quietly (nothing to read)
   if (line.empty())
      continue;   // jump straight to the next loop iteration

   // Copy the line into a string stream. This lets us test the
   // line separately, so errors never damage the main file stream.
   istringstream iss(line);

   double value;    // will hold the number if the conversion works
   string extra;    // will catch any leftover text after the number

   // Two conditions must both be true for a valid line:
   //  1) (iss >> value)   : a floating-point number was read
   //  2) !(iss >> extra)  : nothing else remains on the line
   // Example: "7.5x" reads 7.5 but 'x' is left over -> invalid
   if (iss >> value && !(iss >> extra))
   {
      // Valid line: print the number and update totals
      cout << "Line " << lineNo << ": valid number = " << value << endl;
      sum += value;     // add to the running total
      validCount++;     // count this valid number
   }
   else
   {
      // Invalid line: show a warning on the screen
      cout << "WARNING: Line " << lineNo
           << " skipped (invalid data: \"" << line << "\")" << endl;

      // Also write the warning to the log file
      logFile << "Line " << lineNo
              << ": invalid data \"" << line << "\"" << endl;

      // Clear the error flags (failbit, etc.) so the stream is back
      // in a good state. This is the "clear the error" step.
      iss.clear();

      // No extra code needed to skip: the next getline() call
      // automatically moves on to the next line of the file.
```

```
    }
  }

  // Display a summary after the whole file has been processed
  cout << "\nValid numbers read: " << validCount << endl;
  cout << "Sum of valid numbers: " << sum << endl;

  // Close both files to release system resources
  inFile.close();
  logFile.close();

  return 0;   // 0 means the program finished successfully
}
```

OUTPUT:

```
Error: could not open numbers.txt

--------------------------------
Process exited after 0.01873 seconds with return value 1
Press any key to continue . . .
```

```
Line 1: valid number = 3.14
Line 2: valid number = 42
WARNING: Line 3 skipped (invalid data: "abc")
WARNING: Line 4 skipped (invalid data: "7.5x")
Line 5: valid number = -0.25
WARNING: Line 6 skipped (invalid data: "2.5 hello")
Line 8: valid number = 1000

Valid numbers read: 4
Sum of valid numbers: 1044.89

--------------------------------
Process exited after 0.01423 seconds with return value 0
Press any key to continue . . .
```

2.

CODE:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

int main()
{
  // Open the text file for reading
```

```
ifstream inFile("mixed.txt");

// Check that the file opened correctly before using it
if (!inFile)
{
   cout << "Error: could not open mixed.txt" << endl;
   return 1;   // stop the program: there is nothing to read
}

string line;            // stores the current line from the file
int lineNo = 0;         // current line number
int printedLines = 0;   // how many valid lines were printed
int skippedLines = 0;   // how many invalid lines were skipped

// LOOP OVER FILE CONTENT:
// Reads one line per iteration until end of file is reached
while (getline(inFile, line))
{
   lineNo++;   // move the line counter forward

   // Turn the line into a stream so we can split it into tokens
   // (tokens are the pieces separated by spaces)
   istringstream lineStream(line);

   string token;             // one space-separated piece of the line
   vector<double> numbers;   // stores the numbers found on this line
   bool lineIsValid = true;  // assume the line is good until proven bad
   bool hasToken = false;    // tracks whether the line had any content

   // INNER LOOP: read the line one token (word/number) at a time
   while (lineStream >> token)
   {
      hasToken = true;   // the line is not blank

      // Put this single token into its own stream for testing
      istringstream tokenStream(token);

      double value;      // will hold the converted number
      char leftover;     // catches any extra characters after the number

      // A token is a valid number only if:
      //  1) a number can be read from it  (tokenStream >> value)
      //  2) no characters remain after it (!(tokenStream >> leftover))
      // Example: "5" is valid, but "5x" or "apples" is not.
      if (tokenStream >> value && !(tokenStream >> leftover))
      {
         numbers.push_back(value);   // save the valid number
      }
      else
      {
         // Invalid text found on this line
         tokenStream.clear();    // clear the stream's error state
```

```
            lineIsValid = false;    // mark the whole line as bad
            break;                  // stop checking the rest of the line
        }
    }

    // If the line had no tokens, it was blank: ignore it entirely
    if (!hasToken)
        continue;

    if (lineIsValid)
    {
        // Every token was a number, so print them all
        cout << "Line " << lineNo << " -> ";
        for (size_t i = 0; i < numbers.size(); i++)
        {
            cout << numbers[i] << " ";   // print each number
        }
        cout << endl;
        printedLines++;   // count this valid line
    }
    else
    {
        // The line contained invalid text: report it and move on.
        // The loop continues, so the program is never halted.
        cout << "Line " << lineNo
             << " skipped (contains invalid text): " << line << endl;
        skippedLines++;   // count this skipped line
    }
}

// Print a summary after the file is fully processed
cout << "\nValid lines: " << printedLines
     << ", Skipped lines: " << skippedLines << endl;

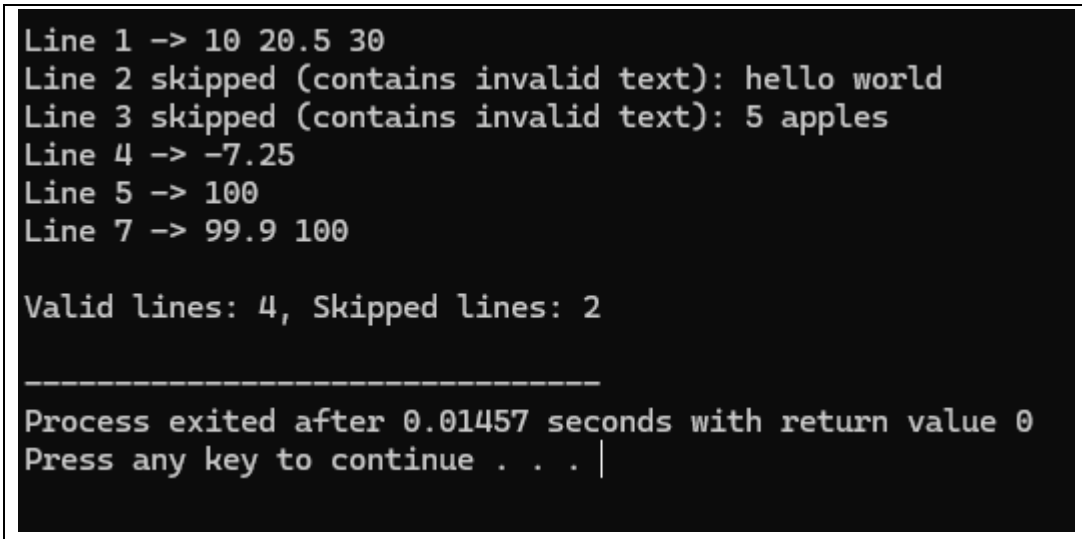
// Close the file to free system resources
inFile.close();

return 0;   // program ended normally
}
```

OUTPUT:

```
Error: could not open mixed.txt

--------------------------------
Process exited after 0.0155 seconds with return value 1
Press any key to continue . . .
```



Conclusion

In this activity, I learned how to open, read, and write files in C++ using ifstream and ofstream, and how looping with getline lets a program process a file line by line until the end. I also learned to handle invalid data safely by using istringstream to test each line and clear() to reset error flags, so the program skips bad input without crashing. Overall, this exercise showed me that good programs should expect messy real-world data and keep running, and it made me more confident in using streams for data processing.